Lojik Devreler ve Tasarım

Bahar 2023

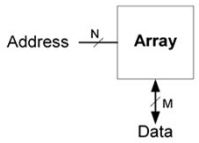
Dr. Öğr. Ü. İhsan Çiçek

10. Bellek Devre Elemanları ve Ardışık Lojik Devre Tasarımı

GEBZE TEKNİK ÜNİVERSİTESİ

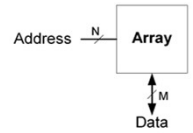
Memory Arrays

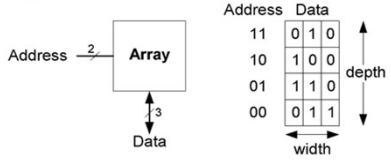
- Efficiently store large amounts of data
- 3 common types:
  - Dynamic random access memory (DRAM)
  - Static random access memory (SRAM)
  - Read only memory (ROM)
- M-bit data value read/ written at each unique N-bit address



Memory Arrays

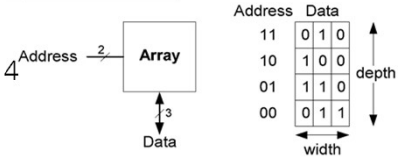
- 2-dimensional array of bit cells
- Each bit cell stores one bit
- N address bits and M data bits:
  - 2^N rows and M columns
  - **Depth:** number of rows (number of words)
  - **Width:** number of columns (size of word)
  - Array size: depth × width = 2^N × M





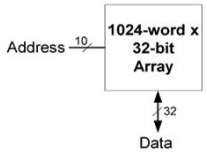
Memory Array Examples

- 2^2 × 3-bit array
- Number of words: 4
- Word size: 3-bits



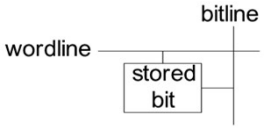
For example: the 3-bit word stored at address 10 is 100

Another example is:



Memory Array Bit Cells

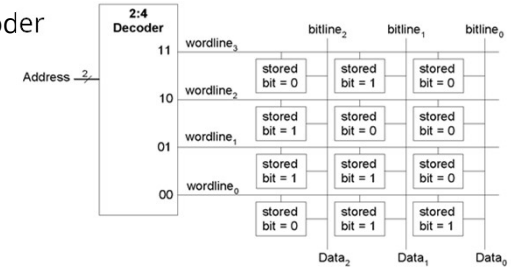
- Memory arrays are built as an **array of bit cells**.
- Each bit cell stores **1 bit** of data
- Connected to a **wordline** and a **bitline**
- When **wordline** is **HIGH**, the stored bit transfers **to or from** the bitline, otherwise the bitline is disconnected
  - **To read:** bitline is left floating (Z), then wordline is turned ON, stored value is transferred to bitline.
  - **To write:** bitline is driven with a logic, then wordline is turned ON, bitline value is transferred to bitcell.



GEBZE 5

Memory Array Bit Cells

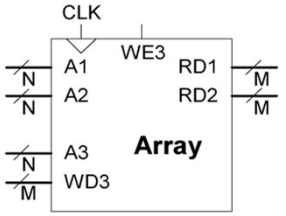
- **Wordline** works like an enable
- **single row** in memory array read/written
- corresponds to **unique** address
- **only one** wordline HIGH at once, driven with a decoder



GEBZE 6

Multi-ported Memories

- All memories have one or more ports.
- Each port gives **read and / or write** access to a memory address.



GEBZE 7

Types of Memory

Memories are classified into two broad categories based on how they store bits in the bit cells.

- **Random access memory (RAM)**
  - **Volatile**: loses its data when power off
  - Read and written quickly
  - Main memory in your computer is RAM (DRAM)
- **Read only memory (ROM)**
  - **Non-volatile**: preserves data when power off
  - Read quickly, but writing is impossible or slow
  - Flash memory in cameras, thumb drives, and digital cameras are all ROMs

GEBZE 8

Random access memory (RAM)

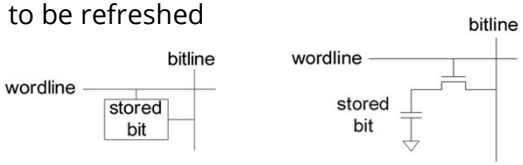
Two major types of RAM:

- DRAM (Dynamic random access memory)
  - uses a capacitor to store the data and a transistor to activate it
  - needs periodic refreshes to preserve the charge
  - slower but less area
- SRAM (Static random access memory)
  - uses cross-coupled inverters and two transistors to active
  - no need to refresh
  - faster but more area

GEBZE 9

DRAM (Dynamic RAM)

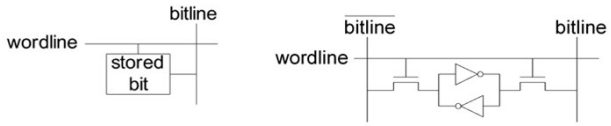
- Data bits stored on **capacitor**
- **Dynamic** because the value needs to be **refreshed** (rewritten) periodically and after read:
  - Charge **leakage** from the capacitor degrades the value
  - Reading **destroys** the stored value, so needs to be refreshed



GEBZE 10

SRAM (Static RAM)

- Data bits stored on **cross-coupled inverters**
- **Static** because the value does NOT need to be refreshed.
- **Noise immune** since inverters will refresh the values.
- More area than the DRAMs, but faster.



GEBZE 11

Area and Delay

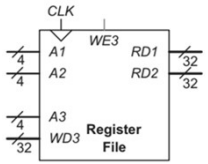
- Flip-flops, SRAMs and DRAMs are all volatile memories, but each have advantages / disadvantages.
- Generally the more transistor means, more area, more power consumption.
- DRAM latency is high because the refresh needs to happen
  - SDRAM and DDR tries to mitigate that shortcomings.

Memory Type	Transistors per Bit Cell	Latency
flip-flop	~20	fast
SRAM	6	medium
DRAM	1	slow

GEBZE 12

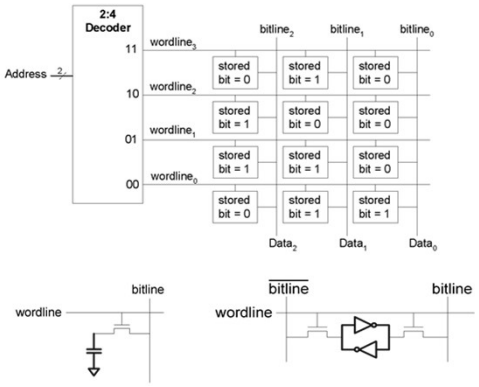
Register Files

- Digital systems often use a number of registers to store temporary variables called registers.
- These group of registers are called a register file.
- Usually built as a small multi-ported SRAM array, because it is more compact than an array of flip-flops



GEBZE TEKNİK ÜNİVERSİTESİ 13

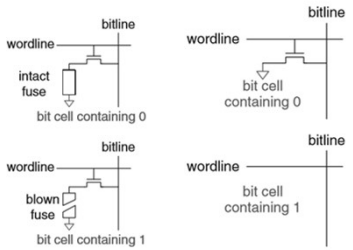
RAM Memory Arrays Review



GEBZE TEKNİK ÜNİVERSİTESİ 14

Read only memory (ROM)

- Non-volatile meaning the values will be there even after a power off.
- Can be thought of as having or not having a transistor.
- It says read-only, but newer types are both read and write (i.e. flash)
- Writing is slower than reading.



GEBZE TEKNİK ÜNİVERSİTESİ 15

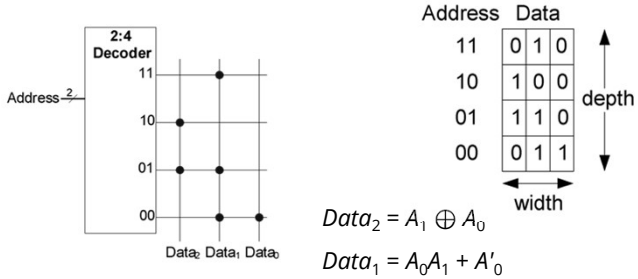
Types of ROMs

- Programmable ROM (PROM)
  - Programmed when manufacturing, (i.e. by blowing up fuses)
  - Contents are fixed, and if mistake, need a new one
  - intended for memory, but used as look-up tables
- Erasable PROMs (EPROM)
  - Floating-gate transistor is used
  - when exposed to ultraviolet (UV) light, contents can be erased
- Electrically Erasable PROMs (EEPROM)
  - instead of UV, high voltage can erase the chip
  - Flash memory is also similar to this

GEBZE TEKNİK ÜNİVERSİTESİ 16

ROM: Dot Notation and Logic

- We can show the ROM contents with DOT notation.
- Can be expressed with boolean logic.



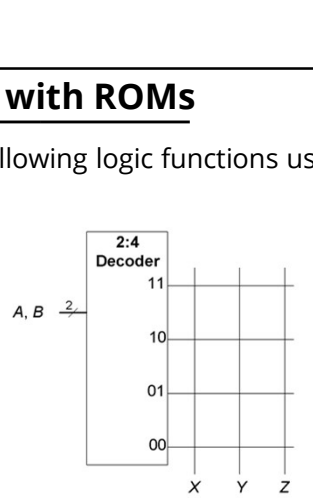
$Data_2 = A_1 \oplus A_0$

$Data_1 = A_0A_1 + A'_0$

$Data_0 = A'_1A'_0$

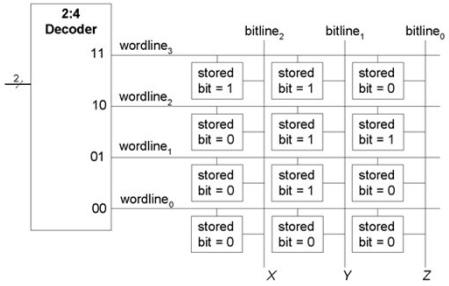
Example: Logic with ROMs

- Implement the following logic functions using a 2^2 × 3-bit ROM:
  - X = AB
  - Y = A + B
  - Z = A B'



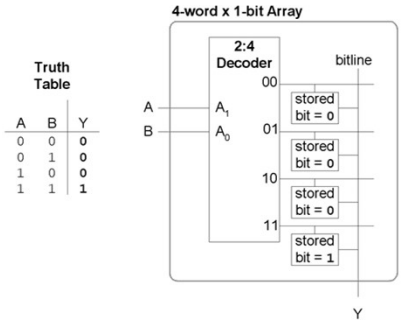
Example: Logic with ROMs

- Implement the following logic functions using a 2^2 × 3-bit ROM:
  - X = AB
  - Y = A + B
  - Z = A B'



Logic with Memory Arrays

Called lookup tables (LUTs): look up output at each input combination (address)

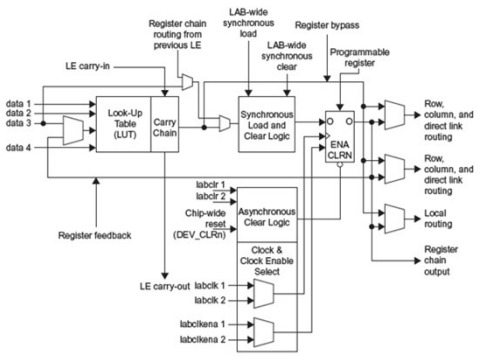


Logic Arrays

- **PLAs** (Programmable logic arrays)
  - AND array followed by OR array
  - Combinational logic only
  - Fixed internal connections
- **FPGAs** (Field programmable gate arrays)
  - Array of Logic Elements (LEs)
  - Combinational and sequential logic
  - Programmable internal connections

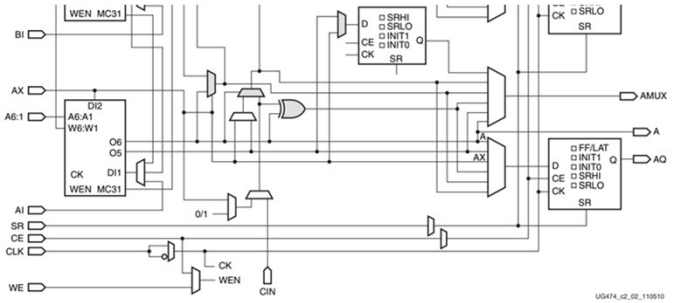
GEBZE TEKNİK ÜNİVERSİTESİ 21

Altera Cyclone IV LE



GEBZE TEKNİK ÜNİVERSİTESİ 22

Xilinx 7-Series Look-up Table



GEBZE TEKNİK ÜNİVERSİTESİ 23

Combinational lojik için always blogu

```
// always @(g1, g2, sel)
always_comb @(g1, g2, sel)
begin
   if (sel) p = g1;
   else     p = g2;
end
```

- always blogu sadece register tanımlamak için kullanılmaz, **aynı zamanda combinational lojik devreleri** de tanımlayabilir
- Yukarıda **g1, g2 ve sel sinyalleriyle aktive** edilen bir always blogu gözükmektedir.
- Burada **else** ifadesi de kullanıldığı ve **sinyal kenarı** belirtilmediği için bir **MUX** sentezleyecektir.

GEBZE TEKNİK ÜNİVERSİTESİ 24

Combinational lojik için always blogu

```
always_comb @(g1, g2, sel)
begin
   if (sel) p = g1;
   else     p = g2;
end
```

```
assign p = sel ? g1 : g2;
```

- Bu iki ifade de aynı anlama gelir.
- sensitivity listede bütün hassas olunan sinyaller mevcut.
- always blogunun içerisinde dikkat edilmesi gereken **her bir dal** için **blok içindeki bütün çıkışlara** atama yapılmasıdır.

GEBZE TEKNİK ÜNİVERSİTESİ

Introduction

- The logic circuits that we have seen so far are **combinational circuits** whose output levels at any instant depend on the current inputs of the system at that time.
- For example, consider a **8-bit 4-to-1 MUX** circuit which decides the **output** based on the select line and the **inputs** to the MUX.

What if we care about the previous computations?

GEBZE TEKNİK ÜNİVERSİTESİ

GEBZE TEKNİK ÜNİVERSİTESİ

Lojik Devreler ve Tasarım

Bahar 2023

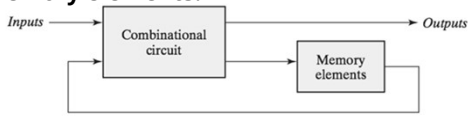
Dr. Öğr. Ü. İhsan Çiçek

10. Bellek Devre Elemanları ve Ardışık Lojik Devre Tasarımı

GEBZE TEKNİK ÜNİVERSİTESİ

Sequential Circuits

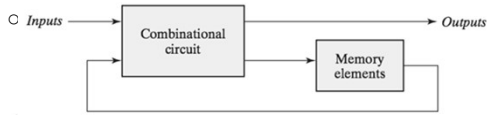
- Imagine a circuit that **increments** a value every time the **user presses a button**, or a **simple security keypad**, where each key press needs to be **remembered** to check for the correct input key pattern.
- These kind of circuits **must have a memory** to **remember its state** in addition to any combinational logic to perform Boolean operations.
- So, the **output of a sequential circuit** depends on the **current inputs** as well as **the previous state of the memory elements.**



GEBZE TEKNİK ÜNİVERSİTESİ

Memory (State) Elements

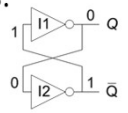
- There are multiple **memory**, **storage** or **state** elements in sequential circuits.
- They are **binary storage devices** capable of storing **one bit** of information.
  - Bistable circuits
  - SR Latch
  - D Latch
  - D Flip-Flop



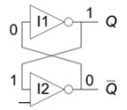
GEBZE

Bistable Circuit Analysis

- Consider the two possible cases:
  - ***Q* = 0:**
    then $Q = 0$, $\bar{Q} = 1$ (consistent)



  - ***Q* = 1:**
    then $Q = 1$, $\bar{Q} = 0$ (consistent)

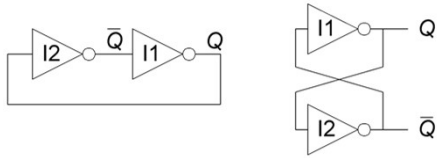


- Stores 1 bit of state in the state variable, Q (or Q)
- But there are **no inputs to control the state**

GEBZE

Bistable Circuit

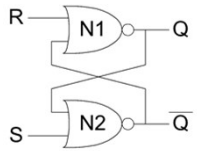
- **Fundamental** building block of other state elements
  - Two outputs: Q, $\bar{Q}$
- **No inputs**



GEBZE

SR (Set/Reset) Latch (NOR)

- SR Latch



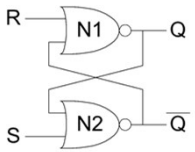
A	B	OR: A + B	NOR: A + B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

- Consider the four possible cases:
  - **Case 1: *S* = 1, *R* = 0**
  - **Case 2: *S* = 0, *R* = 1**
  - **Case 3: *S* = 0, *R* = 0**
  - **Case 4: *S* = 1, *R* = 1**

GEBZE

SR NOR Latch Analysis

- **Case 1: S = 1, R = 0**

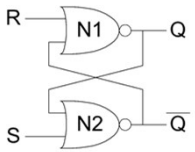


		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

GEBZE TEKNİK ÜNİVERSİTESİ

SR NOR Latch Analysis

- **Case 2: S = 0, R = 1**

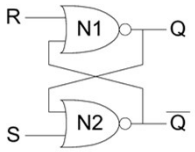


		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

GEBZE TEKNİK ÜNİVERSİTESİ

SR NOR Latch Analysis

- **Case 1: S = 1, R = 0**



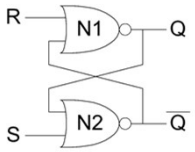
		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

- Stable output
  - **Q = 1**
  - **Q' = 0**
- The output does **not** depend on the previous Q values.
- This is also called **setting** the output

GEBZE TEKNİK ÜNİVERSİTESİ

SR NOR Latch Analysis

- **Case 2: S = 0, R = 1**



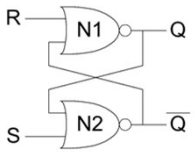
		OR	NOR
A	B	A + B	$\overline{A + B}$
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

- Stable output
  - **Q = 0**
  - **Q' = 1**
- The output does **not** depend on the previous Q values.
- This is also called **resetting** the output

GEBZE TEKNİK ÜNİVERSİTESİ

SR NOR Latch Analysis

- **Case 3: S = 0, R = 0**

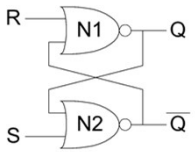


		OR	NOR
A	B	A + B	A + B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

GEBZE

SR NOR Latch Analysis

- **Case 4: S = 1, R = 1**

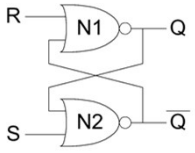


		OR	NOR
A	B	A + B	A + B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

GEBZE

SR NOR Latch Analysis

- **Case 3: S = 0, R = 0**



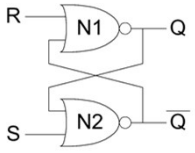
		OR	NOR
A	B	A + B	A + B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

- Stable output
  - if Q = 0 and Q' = 1
    - Q will stay 0
  - if Q = 1 and Q' = 0
    - Q will stay 1
- The output depends on the previous Q values.
- The output does not change! Remembers previous value. **Memory**!

GEBZE

SR NOR Latch Analysis

- **Case 4: S = 1, R = 1**



		OR	NOR
A	B	A + B	A + B
0	0	0	1
0	1	1	0
1	0	1	0
1	1	1	0

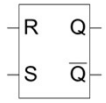
- Since both inputs are high, both outputs will be in 0 state which is not desired!
- This is also called the **forbidden** state!
- **We need a mechanism to avoid this.**

GEBZE

SR NOR Latch Summary

- SR stands for Set/Reset Latch
  - Stores one bit of state (Q)
- Control what value is being stored with S, R inputs
  - **Set:** Make the output 1 ($S = 1$, $R = 0$, $Q = \mathbf{1}$)
  - **Reset:** Make the output 0 ($S = 0$, $R = 1$, $Q = \mathbf{0}$)
- **Must do something to avoid invalid state (when $S = R = 1$)**

SR Latch Symbol

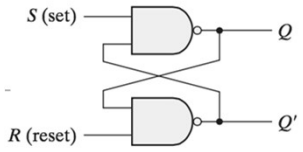


S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

GEBZE TEKNİK ÜNİVERSİTESİ

SR NAND Latch

- Build with two cross-coupled NAND gates instead.
- Works exactly opposite of NOR latch.
- Stores one bit of state (Q)

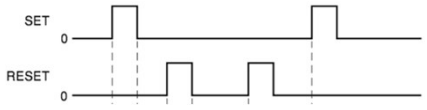


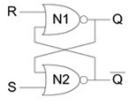
S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after $S = 1, R = 0$)
0	1	1	0	
1	1	1	0	(after $S = 0, R = 1$)
0	0	1	1	(forbidden)

GEBZE TEKNİK ÜNİVERSİTESİ

SR NOR Latch Example

Q. Following inputs are applied to SR NOR Latch. Determine the Q and Q' outputs. (Assume Q = 0 initially)



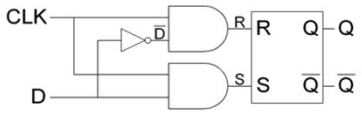


S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

GEBZE TEKNİK ÜNİVERSİTESİ

D Latch

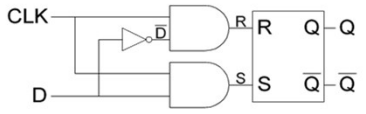
- We can take the SR Latch (NOR) and connect AND gates to the inputs to solve the forbidden state problem.
- Remember, we do **not** want both S and R inputs to go **high** for forbidden state.
- **D and D'** inputs ensures that **only one AND** gate is **active** at any given time.
- **CLK** signal is used to **enable** or **disable** the circuit. That is when it is low, both AND gates are inactive, and when it is high, both AND gates are active.



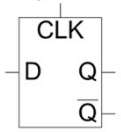
GEBZE TEKNİK ÜNİVERSİTESİ

D Latch Inputs

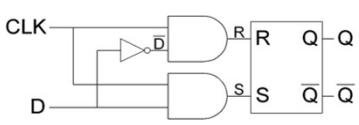
- So we can say that
  - **CLK** controls when the output changes
  - **D** (the data input) controls what the output changes to
- Function
  - When CLK = 1, D passes through to Q (transparent)
  - When CLK = 0, Q holds its previous value

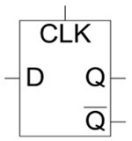


D Latch Symbol



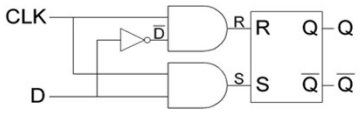
D Latch Truth Table

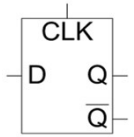




CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	X	$\overline{X}$	0	0	Q_{prev}	$\overline{Q}_{prev}$
1	0	1	0	1	0	1
1	1	0	1	0	1	0

D Latch Truth Table

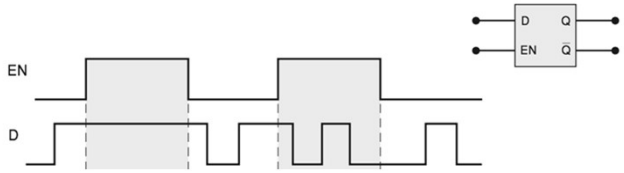




CLK	D	$\overline{D}$	S	R	Q	$\overline{Q}$
0	X					
1	0					
1	1					

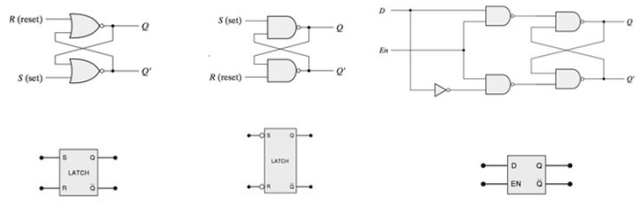
Example

Q. Determine the Q waveform for a D Latch with EN/CLK and D inputs shown in the Figure below. (Assume Q = 0 initially.)



Summary of the types of Latches

SR Latch (NOR) SR Latch (NAND) D Latch



S	R	Q	Q'	
1	0	1	0	
0	0	1	0	(after $S = 1, R = 0$)
0	1	0	1	
0	0	0	1	(after $S = 0, R = 1$)
1	1	0	0	(forbidden)

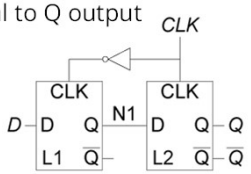
S	R	Q	Q'	
1	0	0	1	
1	1	0	1	(after $S = 1, R = 0$)
0	1	1	0	
1	1	1	0	(after $S = 0, R = 1$)
0	0	1	1	(forbidden)

En	D	Next state of Q
0	X	No change
1	0	$Q = 0$; reset state
1	1	$Q = 1$; set state

GEBZE TEKNİK ÜNİVERSİTESİ

D Flip-Flop

- We can combine two D Latches back to back and tie them with a **complementary** CLK input so that **only one of them is active** in any given time.
- When the CLK signal is LOW the first latch (**L1**) is **active**, and second latch (**L2**) is **not active**.
- Since L1 is active the signal on D input gets passed to the N1 wire.
- As soon as the CLK signal goes **from LOW to HIGH,** L1 deactivates leaving N1 with the latest D value, and passes that signal to Q output



GEBZE TEKNİK ÜNİVERSİTESİ

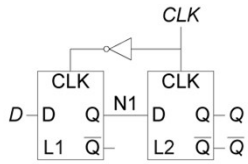
Remarks about Latch

- As we have seen in the example, when the **latch** is **enabled** (CLK/EN input is high) **the input gets copied to the output**.
- We know that the signals arrive at different times through combinational logic elements (delay), and it might not be a good idea to copy the input to the output for a period.
- Therefore, it is **desirable** to create a circuit element where the input is copied to the output **at a single instant** so that we can account for all the delays and activate whenever all the signals are settled.
- This circuit is called a **flip-flop**.

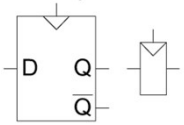
GEBZE TEKNİK ÜNİVERSİTESİ

D Flip-Flop

- Samples D on **rising** or **falling edge** of CLK
- When CLK changes, D passes through to Q, otherwise, Q holds its previous value
- Q changes only on the selected edge of CLK
- Called **edge-triggered**
- Based on which one we connect the CLK input, we can make the flip-flop active on **rising-edge** or **falling-edge** of the CLK.



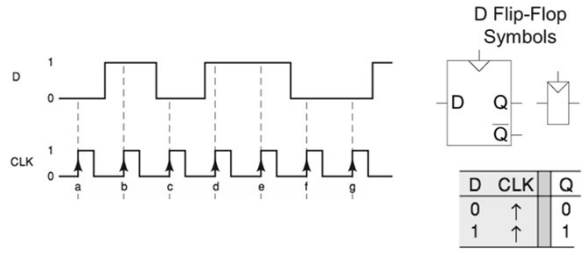
D Flip-Flop Symbols



GEBZE TEKNİK ÜNİVERSİTESİ

Example

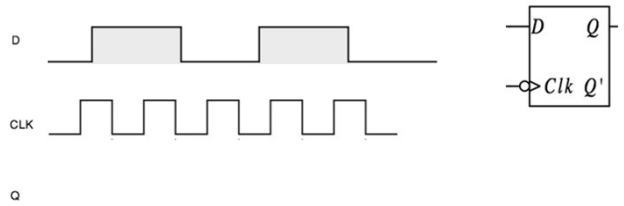
Q. Draw the Q output for the **rising-edge triggered D Flip-Flop**. Assume Q output is 1 initially.



GEBZE

Example

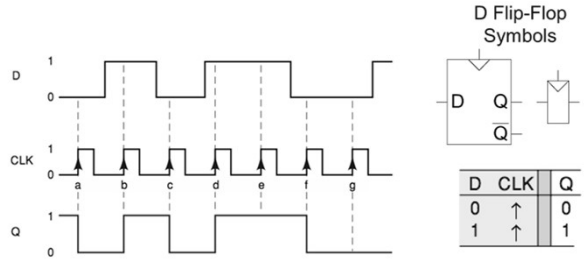
Q. Draw the Q output for the **falling-edge triggered D Flip-Flop**. Assume Q output is 0 initially.



GEBZE

Example

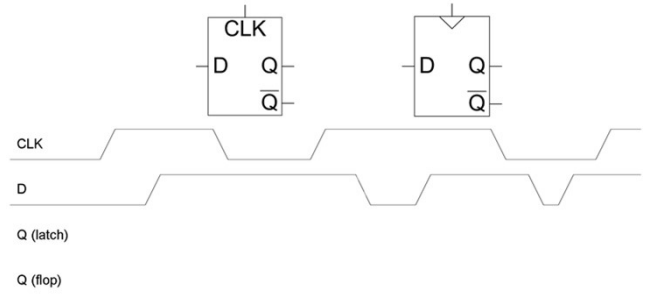
Q. Draw the Q output for the **rising-edge triggered D Flip-Flop**. Assume Q output is 1 initially.



GEBZE

D Latch vs. D Flip-Flop

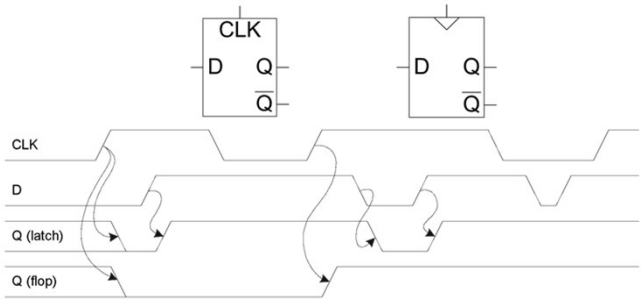
Q. Draw the Q output for the **rising-edge triggered D Flip-Flop** and a **D Latch**. Assume Q outputs are 0 initially.



GEBZE

D Latch vs. D Flip-Flop

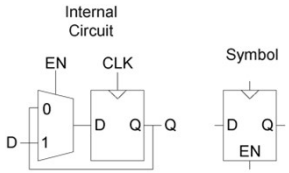
Q. Draw the Q output for the **rising-edge triggered D Flip-Flop** and a **D Latch**. Assume Q outputs are 0 initially.



GEBZE TEKNİK ÜNİVERSİTESİ

Enabled Flip-Flops

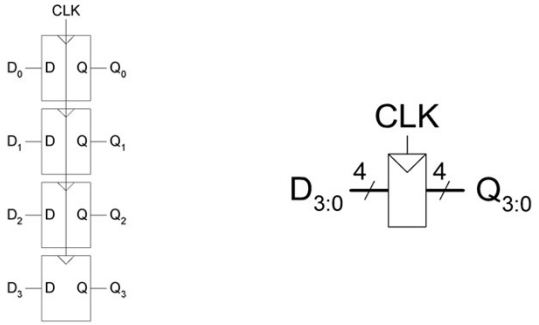
- Sometimes, a Flip-Flop with an enable feature is desirable.
- If the FF is **enabled**, it performs **normally**
- If the FF is **not enabled**, the output **stays the same**.



GEBZE TEKNİK ÜNİVERSİTESİ

Registers: Multi-bit Flip-Flop

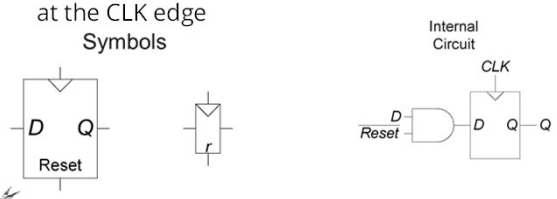
- If we combine multiple Flip-Flops together, we can store as many bits as we want.
- We call this a **Register**



GEBZE TEKNİK ÜNİVERSİTESİ

Resettable Flip-Flops

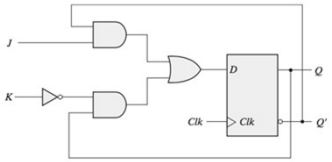
- We can add another input to **reset** the Flip-Flops (Registers)
- There are **two** ways:
  - **Asynchronous reset** - meaning the reset does not wait for a CLK edge.
    - requires changing the internal circuitry of the FF
  - **Synchronous reset** - meaning the reset happens at the CLK edge

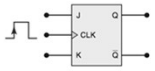


GEBZE TEKNİK ÜNİVERSİTESİ

Bonus: JK Flip-Flop

- It is possible to add the D Flip-Flop to a **set/reset** kind of feature.
- This type of flip-flop is called a **JK Flip-Flop**
  - When J and K inputs are both low the output stays the same
  - When J input is HIGH, the output is HIGH
  - When K input is HIGH, the output is LOW
  - When both J and K inputs are HIGH, the output toggles.





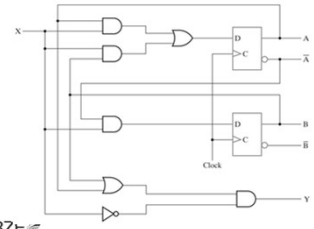
J	K	CLK	Q
0	0	↑	Q_0 (no change)
1	0	↑	1
0	1	↑	0
1	1	↑	$\overline{Q}_0$ (toggles)

GEBZE

Sequential Logic Circuit Analysis

Behavior depends on inputs, outputs and current state
Next state is a function of inputs and current state
FF Input Equations:
Describe the combinational circuits that drive FF inputs

Example:

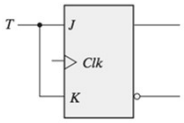


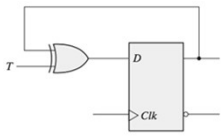
$$D_A = AX + BX$$
$$D_B = \overline{A}X$$
$$Y = (A + B)\overline{X}$$

GEBZE

Bonus: T Flip-Flop

- Another type of Flip-Flop is called a **T Flip-Flop**.
- Its purpose is to **toggle** the output if the T input is **HIGH**, and **keep** the output as is if T input is **LOW**
- We can create a T Flip-Flop using
  - a JK Flip-Flop and tie JK inputs together
  - a D Flip-Flop and tie an XOR gate to the input



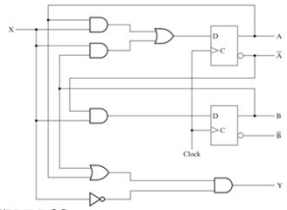


GEBZE

Sequential Logic Circuit Analysis

State Table: An enumerated table showing functional relationship among inputs, outputs and states. m-FFs and n-inputs need 2(m+n) rows in the table which
1. Lists all possible binary combinations of current state and inputs.
2. Next-state values are determined from the FF input equations.
Next states of FF A,B are equal to the present value of their inputs D:
A(t + 1) = DA (t) = AX + BX,
B(t + 1) = DB = A'X,
output Y = AX' + BX'.

$$D_A = AX + BX$$
$$D_B = \overline{A}X$$
$$Y = (A + B)\overline{X}$$



Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

GEBZE

Sequential Logic Circuit Analysis

Next-State section has m-columns for m-FFs. Values derived from the DFF input equations. Output section has the same # of columns for outputs derived from the circuit/Boolean function from the truth table.

Present State		Input	Next State		Output
A	B	X	A	B	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	0	1
0	1	1	1	1	0
1	0	0	0	0	1
1	0	1	1	0	0
1	1	0	0	0	1
1	1	1	1	0	0

Present State		Next State				Output	
		X = 0		X = 1		X = 0	X = 1
A	B	A	B	A	B	Y	Y
0	0	0	0	0	1	0	0
0	1	0	0	1	1	1	0
1	0	0	0	1	0	1	0
1	1	0	0	1	0	1	0

Mealy type sequential logic circuits : The outputs depend on the inputs and the current states. (2D table is used for the outputs as they depend on the inputs)

Moore Type Sequential Logic Circuits: The outputs depend only on the current states (A 1D column table suffices)

GEBZE

Sequential Logic in SystemVerilog

- SystemVerilog uses **idioms** to describe latches, flip-flops and FSMs
- Other coding styles may simulate correctly but produce incorrect hardware
- General Structure:

```
always @(sensitivity list)
  statement;
```

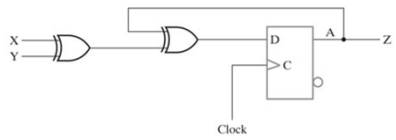
Whenever the event in sensitivity list occurs, statement is executed

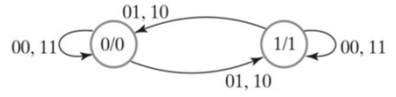
GEBZE 67

Sequential Logic Circuit Analysis

EXAMPLE: Moore model $D_A = A \oplus X \oplus Y \quad Z = A$

The D_A symbol implies a DFF with output designated by the letter A. The X and Y variables are taken as inputs and Z as the output.





Present State	Inputs		Next State	Output
A	X	Y	A	Z
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	0
1	0	0	1	1
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

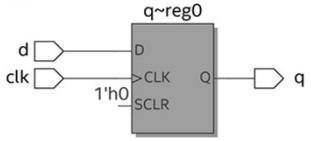
GEBZE

D Flip-Flop

```
module flop(input  logic clk,
            input  logic d,
            output logic q);

always @(posedge clk)
    q <= d;

endmodule
```



- **always** blogunun yanındaki **parantez** içine **sensitivity list** denir.
- Bu listede tanımlanan herhangi bir durum olursa blogun altındaki atamalar gerçekleşir.

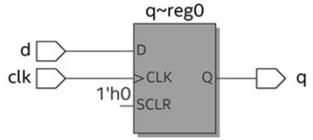
GEBZE 68

D Flip-Flop

```
module flop(input  logic clk,
            input  logic d,
            output logic q);

always @(posedge clk)
    q <= d;

endmodule
```



- **posedge** 0 dan 1 e olan değişimi belirtir. (rising edge)
- **negedge** 1 den 0 a olan değişimi belirtir. (falling edge)
- Yani bir alt satırdaki atama, clk sinyali 0 dan 1 e değiştiği an yapılacaktır. Bu da sentezleyiciye **flip-flop** oluşturmasını söyler.

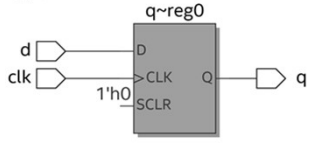
GEBZE 69

D Flip-Flop

```
module flop(input  logic clk,
            input  logic d,
            output logic q);

always @(posedge clk)
    q <= d;

endmodule
```



- **always** blogunun içerisinde **assign** ifadesi kullanılmaz.
- **<=** ifadesi **non-blocking assignment** ı tanımlar (*akıcı atama*)
- **=** ifadesi **blocking assignment** ı tanımlar (*tıkalı atama*)

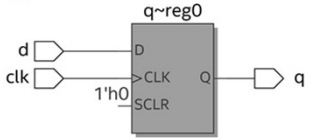
GEBZE 70

D Flip-Flop

```
module flop(input  logic clk,
            input  logic d,
            output logic q);

always @(posedge clk)
    q <= d;

endmodule
```



- **always** blogunun içerisinde combinational logic ve latch de tanımlanabilir.
- Bu yüzden **SystemVerilog always** ifadesini genişleterek üç farklı ifade getirmiştir:
  - **always_ff, always_latch, always_comb**

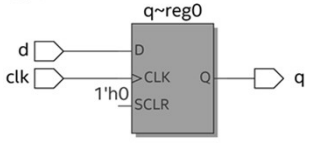
GEBZE 71

D Flip-Flop

```
module flop(input  logic clk,
            input  logic d,
            output logic q);

always_ff @(posedge clk)
    q <= d;

endmodule
```



- Aynı devreyi **always_ff** kullanarak tanımlayabiliriz. Tercih ettiğimiz kullanım bu olacaktır.

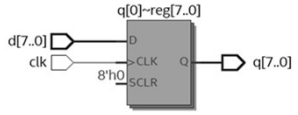
GEBZE 72

Register

```
module reg8(input  logic clk,
            input  logic [7:0] d,
            output logic [7:0] q);

always_ff @(posedge clk)
    q <= d;

endmodule
```



- Eğer giriş ve çıkışlar multi-bit ise her bir bit için ayrı bir flip-flop oluşur.

D Flip-Flop w\ async veya sync reset

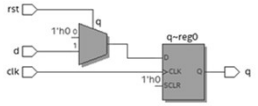
- **reset** sinyali devredeyi bilinen orijinal durumuna getirmek için kullanılır.
- İki farklı tipi vardır:
  - **Asynchronous Reset**
    - reset geldiği anda clk durumuna bakmaksızın işlenir.
    - daha önce bahsettiğimiz gibi **metastability** problemlerine yol açabilir.
    - reset hattında oluşabilecek bir glitch, sistemi resetleyebilir.
  - **Synchronous Reset**
    - reset sinyali clk değişimi olduğu zaman işlenir.
    - sisteme senkron olarak çalışır.

D Flip-Flop w/ synchronous reset

```
module flopsr(input  logic clk, rst,
              input  logic d,
              output logic q);

always_ff @(posedge clk)
    if (!rst) q <= 1'b0;
    else      q <= d;

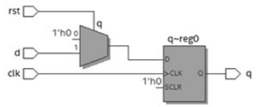
endmodule
```



- **if .. else if .. else** ifadesi always bloklarının altında kullanılabilir.
- sync reset olmasının sebebi, reset sinyalinin sensitivity listesine eklenmemiş olması.

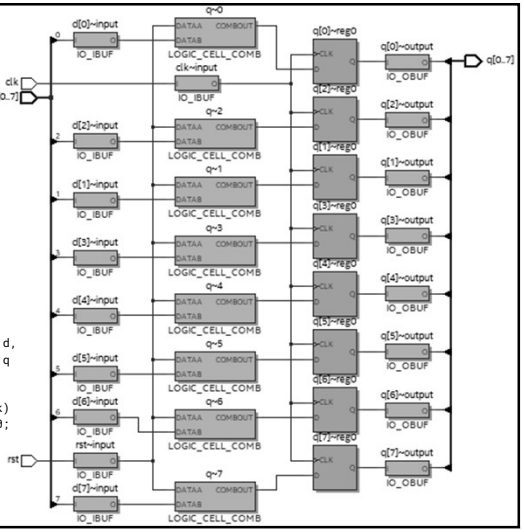
D Flip-Flop w/ synchronous reset

```
...
always_ff @(posedge clk)
    if (!rst) q <= 1'b0;
    else      q <= d;
...
```



- İlk kontrol ettiğimiz sinyal resetin olup olmadığı:
  - reset **aktifse** - q çıkışını sıfırla
  - reset **aktif değilse** - d girişini q çıkışına kopyala
- reset bu örnekte **active-low** olarak bağlı
- Bazı FPGA vendorları sync veya async flip-flop sağlamayabilirler. Bu durumda istenilen fonksiyonu elde etmek için **ekstra lojik** oluşacaktır.
- Kullandığınız FPGA çipi ile alakalı datasheetlere bakmak gerekir.

Intel FPGA 8-bitlik bir register w/ **synch reset** sentezi



```
module reg8sr (
    input  logic clk,
    input  logic rst,
    input  logic [7:0] d,
    output logic [7:0] q
);

always_ff @(posedge clk)
    if (!rst) q <= 8'b0;
    else      q <= d;

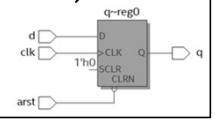
endmodule
```


D Flip-Flop w/ asynchronous reset

```
module flopar(input  logic clk, arst,
              input  logic d,
              output logic q);

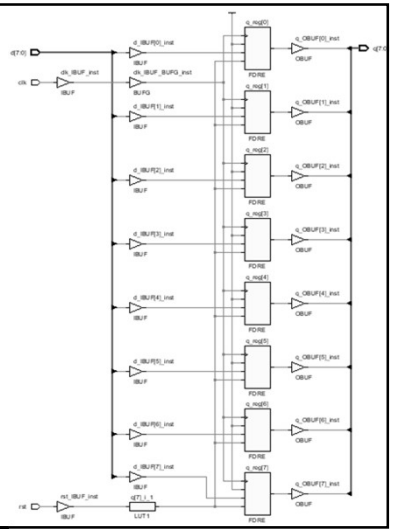
always_ff @(posedge clk, negedge arst)
    if (!arst) q <= 1'b0;
    else       q <= d;

endmodule
```



- async reset olmasının sebebi, reset sinyalinin sensitivity listesine eklenmiş olması!
- **active-low** olarak bağladığımız için reset sinyalinin 1 den 0 a dönüştüğü kenarı ekledik (negedge)

Xilinx FPGA 8-bitlik bir register w/ **synch reset** sentezi



```
module reg8sr (
    input  logic clk,
    input  logic rst,
    input  logic [7:0] d,
    output logic [7:0] q
);

always_ff @(posedge clk)
    if (!rst) q <= 8'b0;
    else      q <= d;

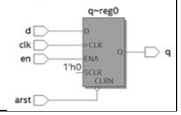
endmodule
```


D Flip-Flop w/ async reset ve enable

```
module floparen(input  logic clk, arst, en,
                input  logic d,
                output logic q);

always_ff @(posedge clk, negedge arst)
    if   (!arst) q <= 1'b0;
    else if (en) q <= d;

endmodule
```



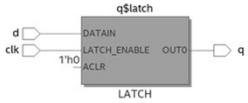
- senkron enable sinyali - genelde ff larda mevcut olur.
- else koşuluna bakmaya gerek yok, çünkü ff enabled değilse çıkışı koruyacak.

D Latch

```
module latchy(input  logic clk,
              input  logic d,
              output logic q);

always_latch
    if (clk) q <= d;

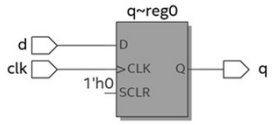
endmodule
```



- **Önemli Not:** Tasarım yaparken bu derste hiçbir zaman latch kullanmayacağız. Yazdığınız kod bazı eksikliklerden dolayı otomatik olarak latch oluşturabilir. Uyarılara dikkat edin ve sentezlenen devreyi kontrol edin. (Özellikle raporlarda gösterecektir.)

Akıcı vs tıkalı ifadeler

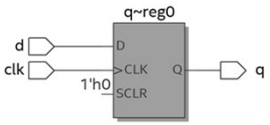
```
always_ff @(posedge clk)
begin
    a = d;
    b = a;
    q = b;
end
```



- **<=** ifadesi **non-blocking** (*akıcı atama*)
- **=** ifadesi **blocking** (*tıkalı atama*)
- Bu devreyi tıkalı atama ile tasarlarsak çok farklı bir sonuç elde edeceğiz.
  - **tıkalı atama** kullandığımız için, bir clock cycleda sistem d yi a'ya, a'yı b'ye ve b'yi de q'ya atamaya çalıştı.
- Sentezleyici otomatik olarak tek flip-flopa düşürdü.

Akıcı vs tıkalı ifadeler

```
always_ff @(posedge clk)
begin
    a = d;
    b = a;
    q = b;
end
```



- Yani sentezleyiciye, clk 0 dan 1 e değişim yaparkeni **d sinyalini a'ya**, **a nın yeni değerini b ye**, **b nin yeni değerini de q ya** atamasını söyledik
- Bütün bu işlem de d sinyalini **direk q ya atamaya eşlenik** olduğundan otomatik olarak **tek** flip flop sentezlendi.
- **always_ff blogu içerisinde her zaman akıcı ifadeler (<=) kullanın.**

Akıcı (<=) vs tıkalı (=) ifadeler

```
always_ff @(posedge clk)
begin
    a <= 4'b1010;
    b <= a;
end
```

- Bütün atamalar always blogunun **sonunda** yapılacak
- clk edge sonrasında
  a = 1010
  b = a nin önceki değeri
- iki ff sentezlenecek

```
always_ff @(posedge clk)
begin
    a = 4'b1010;
    b = a;
end
```

- Atamalar **hemen** yapılacak
- İlk atama yapılasıya kadar o satırda **bekleyecek**
- clk edge sonrasında
  a = 1010
  b = 1010
- tek ff sentezlenecek

Örnek devre 1

```
module ornek1(input  logic       clk,
              input  logic [3:0] d,
              output logic       q);
logic [3:0] f1;
logic f2;

assign f2 = &f1;

always_ff @(posedge clk)
    f1 <= d;

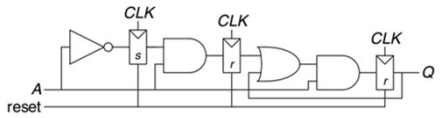
always_ff @(posedge clk)
    q <= f2;

endmodule
```

GEBZE

Örnek devre 2

Aşağıda verilen devreyi tasarlayalım. Bütün resetleri async olarak düşünelim.




```
module ornek2(input  logic clk, reset,
              input  logic A,
              output logic Q);

endmodule
```

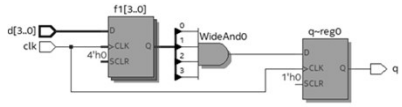
GEBZE

Örnek devre 1

```
always_ff @(posedge clk)
    f1 <= d;

assign f2 = &f1;

always_ff @(posedge clk)
    q <= f2;
```

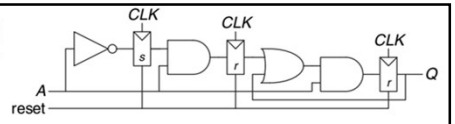


- Tasarımda istediğimiz kadar always blogu bulunabilir.
- Fakat **aynı sinyale farklı always** bloglarında atama **yapamayız**.
- Üç ifadenin sıralamasının bir önemi yok.

GEBZE

86

Örnek devre 2



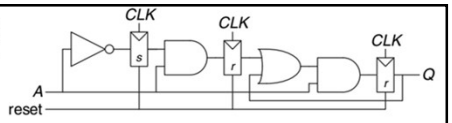

```
logic na, nb, nc;
logic fa, fb, fc;

assign na = ~A;
assign nb = A & fa;
assign nc = (fb | fc) & A;
assign Q = fc;
```

GEBZE

88

Örnek devre 1




```
always_ff @(posedge clk, posedge reset)
begin
    if (reset) begin
        fa <= 1'b0;
        fb <= 1'b0;
        fc <= 1'b0;
    end
    else begin
        fa <= na;
        fb <= nb;
        fc <= nc;
    end
end
```


Örnek devre 2 - birleşik

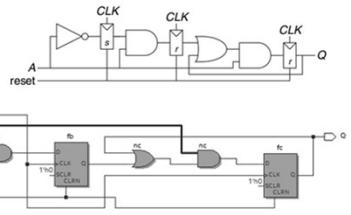
```
module ornek2(input  logic clk, reset,
              input  logic A,
              output logic Q);

logic na, nb, nc;
logic fa, fb, fc;

assign na = ~A;
assign nb = A & fa;
assign nc = (fb | fc) & A;
assign Q = fc;

always_ff @(posedge clk, posedge reset)
begin
    if (reset) begin
        fa <= 1'b0;
        fb <= 1'b0;
        fc <= 1'b0;
    end
    else begin
        fa <= na;
        fb <= nb;
        fc <= nc;
    end
end

endmodule
```

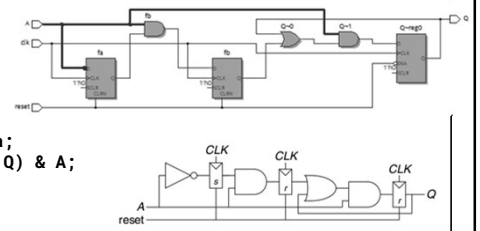


Örnek devre 2 - alternatif

```
module ornek2alt(input  logic clk, reset,
                 input  logic A,
                 output logic Q);
logic fa, fb;

always_ff @(posedge clk, posedge reset)
begin
    if (reset) begin
        fa <= 1'b0;
        fb <= 1'b0;
        Q  <= 1'b0;
    end
    else begin
        fa <= ~A;
        fb <= A & fa;
        Q  <= (fb | Q) & A;
    end
end

endmodule
```



END OF LECTURE

Thanks